

Python — LIST (Part 1)

A list is an ordered, mutable collection of items.

It can store numbers, strings, booleans, objects, or even other lists.

Lists are written with square brackets: []

1. Creating a List

```
numbers = [10, 20, 30, 40]
names = ["Jahanur", "Rahul", "Amit"]
mixed = [10, "Python", True, 3.14]
empty = []
```

KEY IDEA: A list is mutable, so an existing element can be replaced after the list is created.

2. Indexing

Indexing starts at 0. Negative indexing starts at -1.

```
nums = [10, 20, 30, 40, 50]
nums[0]    # 10
nums[2]    # 30
nums[-1]   # 50
nums[-2]   # 40
```

3. Changing Items

```
names = ["A", "B", "C"]
names[1] = "Python"
print(names)
# ["A", "Python", "C"]
```

4. Slicing

Syntax: list[start : stop : step]

The stop index is excluded.

```
a = [10, 20, 30, 40, 50, 60]
a[1:4]    # [20, 30, 40]
a[:3]     # [10, 20, 30]
a[3:]     # [40, 50, 60]
a[::2]    # [10, 30, 50]
a[::-1]   # reverse copy
```

Python — LIST (Part 2)

5. Adding Items

```
a = [10, 20]
a.append(30)      # one item at end
a.insert(1, 15)   # item at index 1
a.extend([40, 50]) # many items
```

6. Removing Items

```
a.remove(20)      # remove value 20
a.pop()           # remove last item
a.pop(1)          # remove index 1
del a[0]          # delete index 0
a.clear()         # remove all items
```

append() adds ONE object. extend() takes an iterable and adds its items.

7. Useful Operations

```
len(a)           # number of items
20 in a          # membership test
a.count(20)       # count occurrences
a.index(20)       # first matching index
sum([1,2,3])     # 6
```

8. Sorting & Reversing

```
nums.sort()      # ascending, in place
nums.sort(reverse=True) # descending
nums.reverse()   # reverse in place
new_nums = sorted(nums) # returns a new list
```

9. Looping Through a List

```
names = ["A", "B", "C"]
for name in names:
    print(name)
```

10. List Comprehension ☆

A compact way to create a new list from an iterable.

```
squares = [x*x for x in range(1, 6)]
# [1, 4, 9, 16, 25]

even = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]
```

Python — LIST (Part 3)

11. Nested Lists

A list can contain other lists. This is useful for tables and matrices.

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
matrix[1][2]    # 6
```

12. Copying Lists — Important!

Using = creates another reference to the same list.

```
a = [1, 2, 3]  
b = a  
b.append(4)  
print(a)        # [1, 2, 3, 4]
```

For a shallow copy, use `copy()` or slicing:

```
b = a.copy()  
# or  
b = a[:]
```

13. Common List Methods — Quick Revision

<code>append(x)</code>	— add x at the end
<code>insert(i,x)</code>	— insert x at index i
<code>extend(it)</code>	— add all items from iterable
<code>remove(x)</code>	— remove first matching value
<code>pop(i)</code>	— remove + return item
<code>clear()</code>	— remove every item
<code>index(x)</code>	— find first index
<code>count(x)</code>	— count occurrences
<code>sort()</code>	— sort in place
<code>reverse()</code>	— reverse in place
<code>copy()</code>	— make shallow copy

14. List vs Tuple

```
my_list = [1, 2, 3]    # mutable  
my_tuple = (1, 2, 3)   # immutable
```

Python — LIST (Part 4)

15. Lists in DSA

Python lists are dynamic arrays. They are very common in coding interviews.

Important complexities to remember:

```
Index access      → O(1)
Append (average)  → O(1)
Search            → O(n)
Insert in middle  → O(n)
Delete in middle  → O(n)
```

16. Useful Built-in Functions

```
len(a)           # length
sum(a)            # total
min(a)           # smallest
max(a)           # largest
sorted(a)        # sorted copy
any(a)           # True if any item is truthy
all(a)           # True if every item is truthy
```

17. Practice Programs ?

1. Find the largest number in a list.
2. Find the smallest number.
3. Calculate sum and average.
4. Count even and odd numbers.
5. Remove duplicates.
6. Reverse a list without reverse().
7. Find the second largest number.
8. Separate positive and negative numbers.
9. Create squares using list comprehension.
10. Find common elements in two lists.
11. Rotate a list by k positions.
12. Find the frequency of each element.

AI/ML connection: data is often organized as rows, columns, arrays, and collections. Strong list skills make NumPy, Pandas, and ML much easier.

Practice every concept with code — understanding comes from doing.